

EJERCICIO 1

1) Diseñar un esquema de base de datos para una escuela usando arquitectura centralizada

Paso 1: Aclarar el contexto (arquitectura centralizada)

En una **arquitectura centralizada**, toda la información se almacena en **un único servidor central**. Los usuarios acceden desde terminales o estaciones de trabajo.

Implicaciones para el diseño:

- Un solo motor DBMS (por ejemplo, PostgreSQL/MySQL) en el servidor central.
- Clientes (secretaría, docentes, dirección) se conectan a ese servidor.
- Debe cuidarse la disponibilidad (es un punto único de falla si no hay redundancia).

Paso 2: Definir el alcance del sistema escolar

Para un esquema base típico, se requiere:

- Registrar estudiantes.
- Registrar docentes.
- Registrar cursos.
- Ofertar cursos por secciones (por periodo/año, aula, horario).
- Matricular estudiantes en secciones.
- Registrar evaluaciones/notas.

Paso 3: Identificar entidades principales

Entidades mínimas:

1. **ESTUDIANTE**
2. **DOCENTE**
3. **CURSO**
4. **SECCION** (curso dictado por un docente en un periodo)
5. **MATRICULA** (inscripción de estudiante en una sección)
6. **EVALUACION** (notas por matrícula)

Paso 4: Definir atributos clave (PK) y de negocio

Ejemplo de atributos:

- ESTUDIANTE: id_estudiante (PK), dni, nombres, apellidos, email, etc.
- DOCENTE: id_docente (PK), dni, nombres, apellidos, especialidad, etc.
- CURSO: id_curso (PK), nombre, nivel, créditos, etc.
- SECCION: id_seccion (PK), id_curso (FK), id_docente (FK), anio, periodo, aula, horario, cupo.

- MATRICULA: id_matricula (PK), id_estudiante (FK), id_seccion (FK), fecha_matricula, estado.
- EVALUACION: id_evaluacion (PK), id_matricula (FK), tipo, fecha, ponderacion, nota.

Paso 5: Definir relaciones y cardinalidades

- CURSO 1..* SECCION (un curso puede abrir varias secciones).
- DOCENTE 1..* SECCION (un docente puede dictar varias secciones).
- ESTUDIANTE 1..* MATRICULA.
- SECCION 1..* MATRICULA.
- MATRICULA 1..* EVALUACION.

Paso 6: Proponer el modelo lógico (tablas y claves)

Tablas sugeridas (lógico):

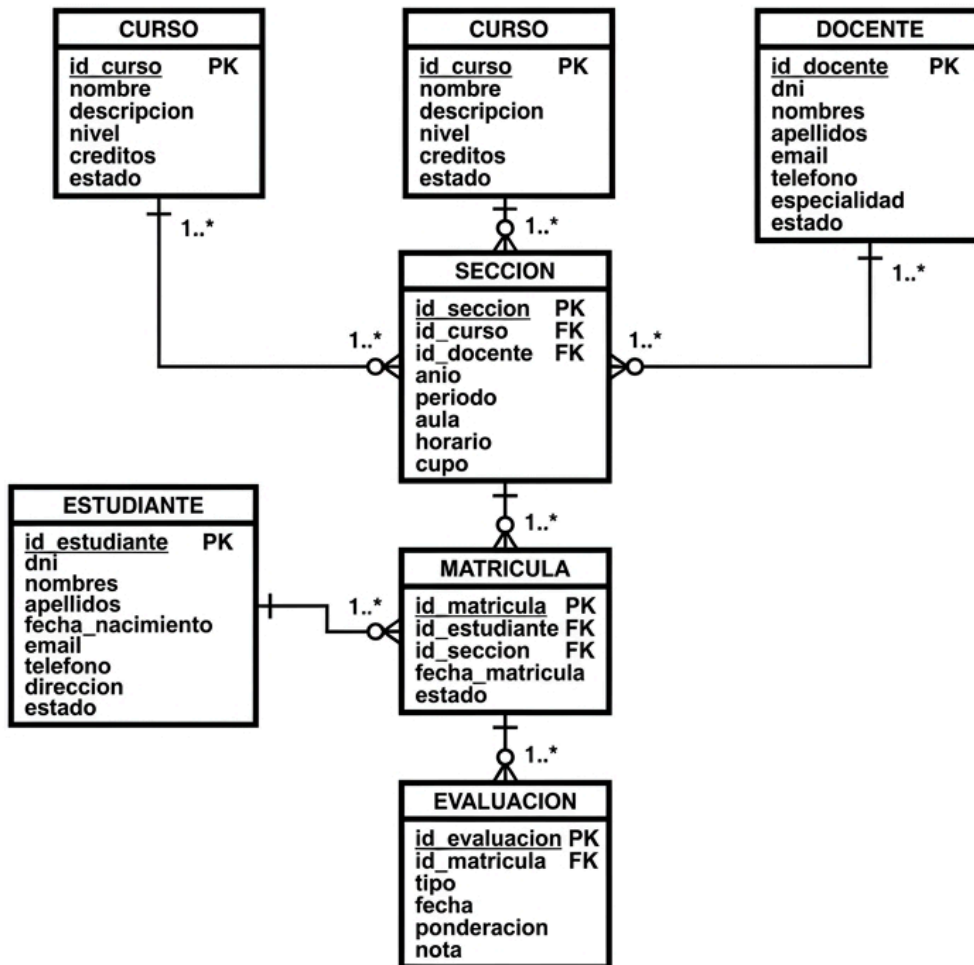
- estudiante(id_estudiante PK, dni UNIQUE, nombres, apellidos, fecha_nacimiento, email UNIQUE, telefono, direccion, estado)
- docente(id_docente PK, dni UNIQUE, nombres, apellidos, email UNIQUE, telefono, especialidad, estado)
- curso(id_curso PK, nombre, descripcion, nivel, credits, estado)
- seccion(id_seccion PK, id_curso FK, id_docente FK, anio, periodo, aula, horario, cupo)
- matricula(id_matricula PK, id_estudiante FK, id_seccion FK, fecha_matricula, estado, UNIQUE(id_estudiante, id_seccion))
- evaluacion(id_evaluacion PK, id_matricula FK, tipo, fecha, ponderacion, nota)

Paso 7: Reglas y restricciones recomendadas

- **UNIQUE(dni)** en estudiante/docente.
- **UNIQUE(email)** si se desea que sea único.
- **CHECK(ponderacion between 0 and 1)** o 0..100 según criterio.
- **CHECK(nota between 0 and 20)** (o escala usada).
- **UNIQUE(id_estudiante, id_seccion)** para evitar doble matrícula.

Paso 8: Consideraciones de centralización (operación)

- RespalDOS periódicos (backup).
- UPS/energía y plan de recuperación.
- Si se requiere alta disponibilidad, agregar réplica o clustering (aunque siga siendo “central” en términos de operación).



2) Comparar ventajas y desventajas de cliente-servidor vs nube para un e-commerce

Paso 1: Definir el caso de uso

E-commerce típico requiere:

- Alta disponibilidad (ventas 24/7).
- Picos de tráfico (campañas, fechas especiales).
- Seguridad (pagos, datos personales).
- Escalabilidad (crecer sin rediseñar todo).

Paso 2: Describir la arquitectura cliente-servidor (on-premise o propio)

Flujo: Clientes/Apps -> Red -> Servidor Web/App -> Servidor BD.

Ventajas (cliente-servidor):

1. Control total de hardware y software.
2. Costos iniciales fijos (CAPEX) si ya se compra infraestructura.
3. Menor latencia local si usuarios están cerca.

4. Seguridad física y control interno.
5. Independencia de proveedor cloud.

Desventajas (cliente-servidor):

1. Escalabilidad limitada o costosa (comprar más servidores).
2. Alto mantenimiento (parches, monitoreo, reemplazos).
3. Punto único de falla si no se implementa HA.
4. Costo operativo alto a largo plazo (personal TI, energía).
5. Menor flexibilidad ante picos.

Paso 3: Describir la arquitectura en la nube

Flujo: Clientes/Apps -> Internet -> Servicios Cloud (CDN/API) -> Balanceo -> Servicios App
-> BD Gestionada + almacenamiento.

Ventajas (nube):

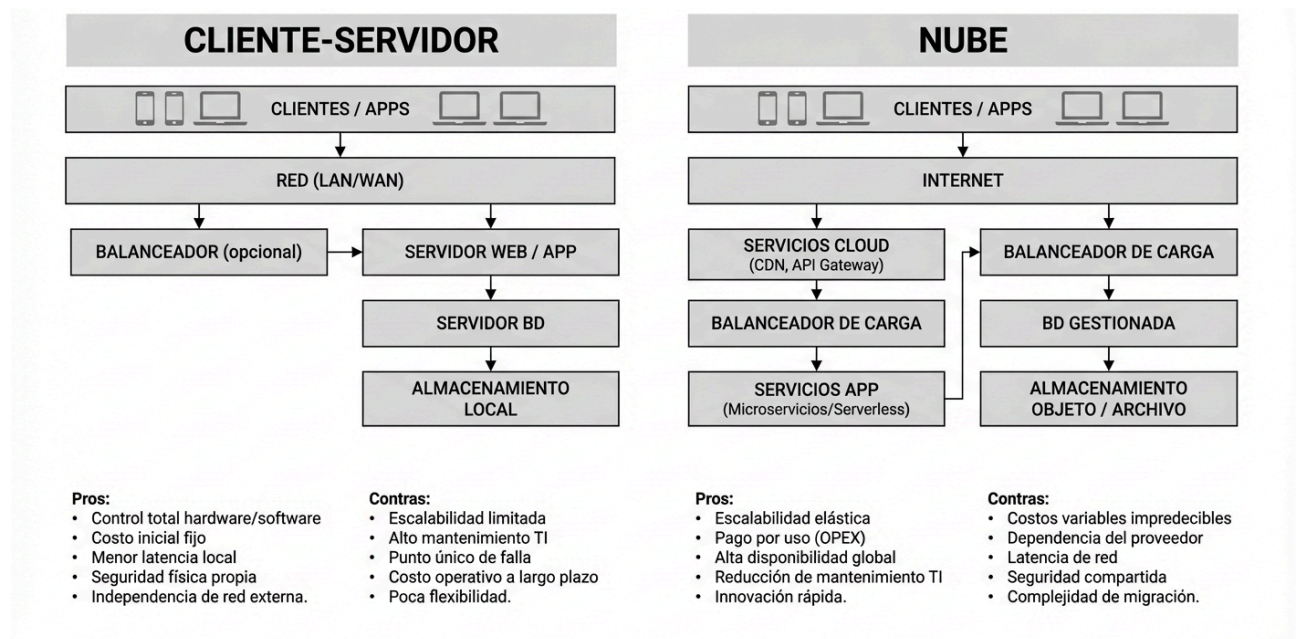
1. Escalabilidad elástica (autoscaling).
2. Pago por uso (OPEX).
3. Alta disponibilidad global (multi-zona/región).
4. Menor carga de mantenimiento TI (servicios gestionados).
5. Iteración rápida (nuevos servicios, despliegues automatizados).

Desventajas (nube):

1. Costos variables impredecibles si no se controla (tráfico/servicios).
2. Dependencia del proveedor.
3. Latencia de red depende de conectividad.
4. Seguridad compartida (responsabilidades divididas).
5. Migración y arquitectura pueden ser complejas.

Paso 4: Conclusión para e-commerce

- Si el e-commerce espera **picos**, crecimiento y operación 24/7, **nube** suele ser la opción más eficiente.
- Si el negocio es pequeño, local y con equipo TI fuerte, cliente-servidor puede ser viable, pero debe contemplar HA, backups y escalamiento.



3) Simular un caso de fallas en una base de datos distribuida y proponer soluciones

Paso 1: Definir escenario distribuido

Una BD distribuida puede tener:

- Réplicas en diferentes sedes.
- Particiones (sharding) por región.
- Servicios que deben mantenerse consistentes.

Paso 2: Proponer una simulación concreta de fallas

Escenario: Empresa con 3 nodos (Lima, Bogotá, México) replicando datos de clientes.

Simulación de falla:

1. Nodo Lima es el primario.
2. Se cae la conectividad entre Lima y los otros nodos (partición de red).
3. Bogotá y México siguen recibiendo escrituras (dependiendo del sistema, pueden quedar en modo lectura o aceptar escrituras con riesgo).
4. Cuando vuelve la red, aparecen conflictos de datos (versiones diferentes).

Paso 3: Identificar impactos

- **Disponibilidad:** usuarios pueden no acceder a datos actualizados.
- **Consistencia:** riesgo de datos divergentes si se aceptan escrituras simultáneas.
- **Latencia:** aumenta si el sistema intenta confirmar en varios nodos.

Paso 4: Proponer soluciones (técnicas)

1. **Estrategia de consistencia definida:**
 - Fuerte consistencia (CP) si la prioridad es exactitud (banca).
 - Alta disponibilidad (AP) si la prioridad es operar aunque haya particiones (catálogos).
2. **Quórum de lecturas/escrituras:**
 - Configurar R/W para que no se confirmen escrituras sin mayoría.
3. **Failover automatizado:**
 - Si cae el primario, promover réplica a primario (con herramientas del DBMS).
4. **Resolución de conflictos:**
 - Last-write-wins (simple, pero puede perder datos).
 - Versionado y conciliación por reglas de negocio.
5. **Monitoreo y alertas:**
 - Métricas de replicación (lag), salud de nodos, latencia.
6. **Backups y recuperación:**
 - Snapshots y pruebas periódicas de restore.

Paso 5: Proponer soluciones (procedimentales)

- Plan de contingencia (runbook) para incidentes.
- Pruebas de “chaos engineering” controladas.
- Documentar RPO/RTO.

4) Analizar qué arquitectura usar en un hospital regional

Paso 1: Identificar necesidades del hospital

- Datos críticos (historia clínica).
- Alta disponibilidad (emergencias).
- Confidencialidad (cumplimiento normativo).
- Uso multiárea (triaje, laboratorio, farmacia, admisión).

Paso 2: Evaluar opciones

Centralizada:

- Pro: control interno.
- Contra: punto único de falla.

Cliente-servidor (local):

- Pro: latencia baja en red interna, control.
- Contra: requiere infraestructura y equipo TI fuerte; HA debe implementarse.

Distribuida:

- Pro: tolerancia a fallas y sedes.
- Contra: complejidad alta (sincronización, consistencia).

Nube (o híbrida):

- Pro: alta disponibilidad, DR, escalabilidad.
- Contra: dependencia de internet, controles de compliance.

Paso 3: Recomendación

Para un hospital regional, la mejor práctica suele ser:

- **Arquitectura híbrida:**
 - Operación local (cliente-servidor) para continuidad.
 - Réplica/backup en nube para recuperación ante desastres.

Paso 4: Justificación

- Asegura operación aun con caídas parciales de conectividad.
- Permite DR (disaster recovery) más robusto.
- Reduce riesgo de pérdida de información.

5) Proponer criterios de selección para una startup de software educativo

Paso 1: Contexto típico de startup

- Presupuesto ajustado.
- Necesidad de escalar rápido.
- Equipo TI pequeño.

Paso 2: Criterios de selección (propuesta)

1. **Escalabilidad:** capacidad de crecer por demanda.
2. **Costo total:** CAPEX vs OPEX, previsibilidad.
3. **Velocidad de entrega:** despliegue rápido, automatización.

4. **Seguridad:** protección de datos de menores, cifrado.
5. **Disponibilidad:** tolerancia a fallas.
6. **Mantenibilidad:** complejidad operativa.
7. **Cumplimiento:** normativas de protección de datos.

Paso 3: Conclusión

- Generalmente, una startup educativo se beneficia de **nube** (servicios gestionados) por velocidad y escalamiento, cuidando costos con monitoreo y presupuestos.
-

6) Determinar la mejor arquitectura para una empresa con sucursales internacionales

Paso 1: Identificar el reto

- Usuarios en diferentes países.
- Latencia si todo está en un solo servidor.
- Necesidad de continuidad ante fallas regionales.

Paso 2: Opciones

- Centralizada: latencia alta y riesgo.
- Cliente-servidor local por país: fragmenta la información.
- Distribuida o nube multi-región: mejor equilibrio.

Paso 3: Recomendación

- **Nube multi-región o BD distribuida** (según DBMS y requisitos de consistencia).

Paso 4: Justificación

- Menor latencia (datos más cerca del usuario).
 - Alta disponibilidad global.
 - Escalabilidad por regiones.
-

7) Crear un diagrama E-R de un sistema bancario

Paso 1: Identificar entidades centrales

- CLIENTE
- CUENTA
- TRANSACCION

- PRESTAMO
- PAGO_PRESTAMO
- SUCURSAL
- (Tabla puente) CUENTA_SUCURSAL

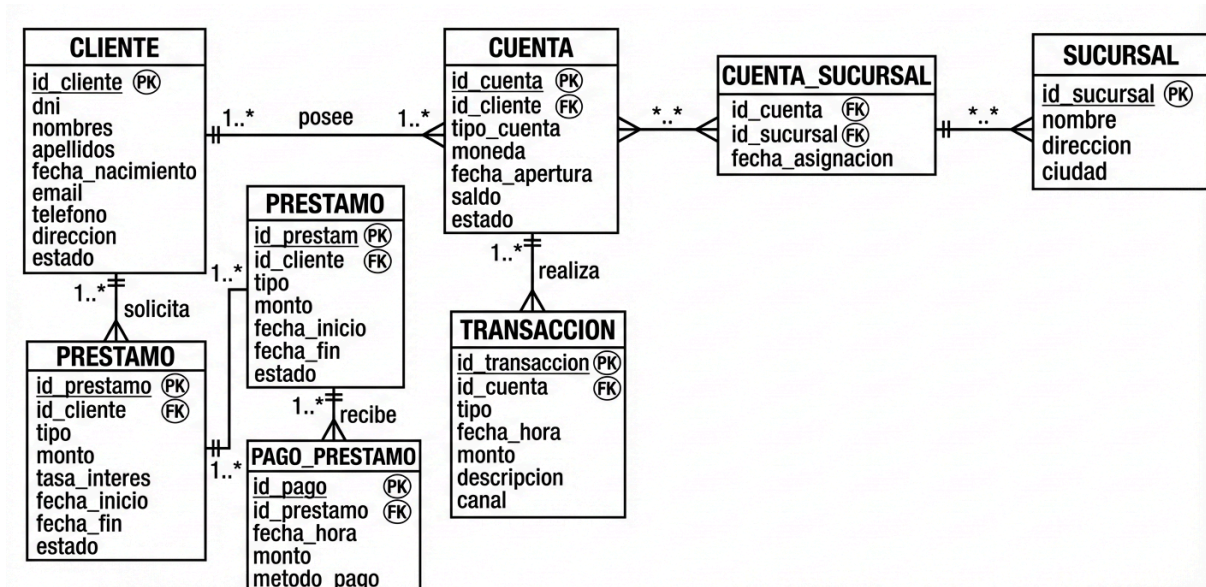
Paso 2: Definir relaciones

- CLIENTE 1..* CUENTA
- CUENTA 1..* TRANSACCION
- CLIENTE 1..* PRESTAMO
- PRESTAMO 1..* PAGO_PRESTAMO
- CUENTA .. SUCURSAL mediante CUENTA_SUCURSAL

Paso 3: Validar reglas de negocio

- Un cliente puede tener varias cuentas.
- Una cuenta registra muchas transacciones.
- Un préstamo recibe muchos pagos.
- Una cuenta puede asociarse a una o varias sucursales (según administración/portabilidad).

Paso 4: Resultado



8) Transformar modelo conceptual de un hospital a modelo lógico

Paso 1: Definir un conceptual mínimo (ejemplo)

Entidades conceptuales típicas:

- PACIENTE
- MEDICO

- CITA
- HISTORIA_CLINICA
- DIAGNOSTICO
- RECETA
- MEDICAMENTO

Relaciones conceptuales (resumen):

- PACIENTE 1..* CITA
- MEDICO 1..* CITA
- PACIENTE 1..1 HISTORIA_CLINICA
- HISTORIA_CLINICA 1..* DIAGNOSTICO
- DIAGNOSTICO 1..* RECETA
- RECETA .. MEDICAMENTO (tabla puente)

Paso 2: Pasar a tablas (lógico)

1. Cada entidad fuerte se convierte en tabla:
 - paciente(id_paciente PK, dni UNIQUE, nombres, apellidos, fecha_nacimiento, sexo, telefono, email, direccion)
 - medico(id_medico PK, cmp UNIQUE, nombres, apellidos, especialidad, telefono, email)
2. Relación 1..* se implementa con FK en el lado “muchos”:
 - cita(id_cita PK, id_paciente FK, id_medico FK, fecha_hora, motivo, estado)
3. Relación 1..1 se implementa con FK UNIQUE o PK compartida:
 - historia_clinica(id_historia PK, id_paciente FK UNIQUE, fecha_apertura, observaciones)
4. Relación .. requiere tabla puente:
 - receta_medicamento(id_receta FK, id_medicamento FK, dosis, frecuencia, duracion, PRIMARY KEY(id_receta, id_medicamento))

Paso 3: Definir restricciones

- NOT NULL en campos obligatorios.
- CHECK en estados (catálogos).
- Índices en campos de búsqueda (dni, cmp, fecha_hora).

Paso 4: Validación rápida

- Probar inserciones: crear paciente, médico, cita, historia, diagnóstico.
- Probar integridad: no permitir cita con id_paciente inexistente.

9) Validar relaciones y restricciones en un DBMS con datos de prueba

Paso 1: Crear tablas con constraints

- PK, FK, UNIQUE, CHECK.

Paso 2: Insertar datos válidos

Ejemplo (escuela):

1. Insertar curso, docente, estudiante.
2. Insertar sección con FKs válidas.
3. Insertar matrícula.
4. Insertar evaluación.

Paso 3: Probar casos inválidos (debe fallar)

1. Matrícula duplicada (mismo estudiante y sección) si existe UNIQUE.
2. Evaluación con id_matricula inexistente (FK).
3. Nota fuera de rango (CHECK).

Paso 4: Documentar evidencias

- Capturas o logs de error.
- Consulta final confirmando datos consistentes.

10) Normalizar la base de datos de un sistema de inventario hasta 3FN

Paso 1: Partir de un ejemplo “no normalizado” (0FN)

Supongamos una tabla inicial (mal diseñada) para inventario/ventas:

VENTA_RAW

id_venta

fecha

cliente_nombre

cliente_dni

cliente_telefono

productos (lista dentro de una celda, ej: “P01:Mouse:2:50; P02:Teclado:1:90”)

total

Problemas:

Repetición de datos de cliente.

“productos” no es atómico (viola 1FN).

Difícil calcular stock por producto, etc.

Paso 2: Llevar a 1FN (atributos atómicos)

Separar los productos en filas, no en una lista:

VENTA_DETALLE_1FN

id_venta

fecha

cliente_dni

cliente_nombre

cliente_telefono

id_producto

nombre_producto

cantidad

precio_unitario

Aún hay redundancia:

Datos de cliente repetidos por cada producto vendido.

Datos de producto repetidos por cada venta.

Paso 3: Llevar a 2FN (dependencias completas de la clave)

Si la clave compuesta es (id_venta, id_producto), entonces:

fecha depende solo de id_venta

cliente_dni/nombre/teléfono dependen solo de id_venta (o del cliente)

nombre_producto depende solo de id_producto

Separamos en tablas:

VENTA

id_venta (PK)

fecha

id_cliente (FK)

total (opcional, puede calcularse)

DETALLE_VENTA

id_venta (PK, FK)

id_producto (PK, FK)

cantidad

precio_unitario

CLIENTE

id_cliente (PK)

dni (UNIQUE)

nombre

telefono

PRODUCTO

id_producto (PK)

nombre

precio_referencial

(otros)

Paso 4: Llevar a 3FN (sin dependencias transitivas)

Revisar dependencias transitivas, por ejemplo:

Si PRODUCTO tiene id_categoria y categoría_nombre en la misma tabla, categoría_nombre depende de id_categoria, no del producto directamente.

Entonces:

CATEGORIA

id_categoria (PK)

nombre

Y PRODUCTO queda:

id_producto (PK)

nombre

id_categoria (FK)

precio_referencial

Paso 5: Tablas recomendadas para inventario

Para stock y movimientos:

ALMACEN

id_almacen (PK)

nombre

ubicacion

STOCK

id_almacen (PK, FK)

id_producto (PK, FK)

cantidad_actual

MOVIMIENTO_INVENTARIO

id_movimiento (PK)

id_almacen (FK)

id_producto (FK)

tipo (ENTRADA|SALIDA|AJUSTE)

fecha_hora

cantidad

referencia (compra/venta/ajuste)

Resultado:

3FN lograda con datos sin redundancia innecesaria.

Consultas y mantenimiento más consistentes.

11) Crear índices para mejorar consultas en una base de datos de ventas

Paso 1: Identificar consultas frecuentes (ejemplos)

Ventas por rango de fechas:

```
SELECT * FROM venta
```

```
WHERE fecha BETWEEN '2026-01-01' AND '2026-01-31';
```

Ventas por cliente:

```
SELECT * FROM venta
```

```
WHERE id_cliente = 10;
```

Detalle por venta (join):

```
SELECT dv.*
```

```
FROM detalle_venta dv
```

```
WHERE dv.id_venta = 1001;
```

Ranking de productos (agregación):

```
SELECT id_producto, SUM(cantidad)
```

FROM detalle_venta

GROUP BY id_producto;

Paso 2: Proponer índices (y justificar)

Índice por fecha en VENTA:

Mejora filtros por rango.

INDEX(fecha)

Índice por id_cliente en VENTA:

Mejora consultas por cliente.

INDEX(id_cliente)

Índice compuesto en DETALLE_VENTA:

Si PK es (id_venta, id_producto), ya sirve para buscar por id_venta.

Si la PK no existe, crear:

INDEX(id_venta) o INDEX(id_venta, id_producto)

Índice por id_producto en DETALLE_VENTA:

Mejora agregación por producto (dependiendo del motor y el plan).

INDEX(id_producto)

Paso 3: Evitar exceso de índices

Cada índice cuesta espacio y ralentiza INSERT/UPDATE.

Mantener índices alineados a consultas reales.

Paso 4: Validación

Medir con EXPLAIN (o plan de ejecución) antes y después.

Comparar tiempo y lecturas.

12) Decidir qué tablas conviene desnormalizar en un sistema de reportes

Paso 1: Distinguir OLTP vs Reporting

OLTP (operación diaria): conviene normalizar (consistencia).

Reporting/BI: conviene optimizar lectura, a veces desnormalizando.

Paso 2: Candidatos típicos a desnormalizar

Hechos de ventas: se consulta muchísimo por fecha, producto, categoría, cliente.

Dimensiones (producto, categoría, cliente): a veces se “aplanan” para reportes.

Paso 3: Ejemplo de desnormalización (reporte)

En vez de hacer muchos JOIN:

VENTA_REPORTER (tabla materializada o vista materializada)

id_venta

fecha

id_cliente

cliente_nombre

id_producto

producto_nombre

categoria_nombre

cantidad

precio_unitario

subtotal

Ventajas:

Reportes más rápidos.

Menos joins.

Desventajas:

Redundancia.

Necesitas proceso ETL/actualización (por lotes o incremental).

Paso 4: Criterio de decisión

Desnormalizar cuando:

Es “solo lectura” para reportes.

Hay consultas repetitivas y pesadas.

Aceptas una actualización diferida (por ejemplo cada 15 min, cada noche).

Mantener normalizado cuando:

Es transaccional y crítico (banca, stock en tiempo real).

13) Realizar entrevistas simuladas para extraer requerimientos (hospital)

Paso 1: Definir roles a entrevistar

- Admisión/Recepción
- Médicos
- Enfermería
- Laboratorio
- Farmacia
- TI/Seguridad

Paso 2: Preparar guion de preguntas

Ejemplos:

1. ¿Qué datos se registran al ingresar un paciente?

2. ¿Cómo se agenda una cita y quién la aprueba?
3. ¿Qué información debe estar disponible en emergencias?
4. ¿Cómo se controlan accesos por perfil?
5. ¿Qué reportes se necesitan (diarios, semanales, auditoría)?

Paso 3: Ejecutar entrevistas simuladas (estructura)

1. Presentación del objetivo.
2. Preguntas abiertas.
3. Profundizar con “¿qué pasa si...?” (excepciones).
4. Confirmar flujos con ejemplos reales.
5. Cerrar con resumen y validación.

Paso 4: Consolidar hallazgos

- Lista de procesos.
- Mapa de actores.
- Diccionario de datos inicial.

14) Lista de requerimientos funcionales y no funcionales (biblioteca digital)

Paso 1: Alcance

Sistema para préstamos digitales y gestión de catálogo.

Paso 2: Requerimientos funcionales (RF)

1. RF1: Registrar usuarios (alta/baja/edición).
2. RF2: Autenticación e inicio de sesión.
3. RF3: Buscar libros por título/autor/categoría.
4. RF4: Ver disponibilidad de ejemplares o licencias.
5. RF5: Reservar libros.
6. RF6: Prestar libros (generar préstamo).
7. RF7: Devolver libros (cerrar préstamo).
8. RF8: Notificar vencimientos (correo o app).
9. RF9: Gestionar catálogo (admin).
10. RF10: Reportes (más prestados, morosidad, actividad).

Paso 3: Requerimientos no funcionales (RNF)

1. RNF1 (Disponibilidad): 99.5% o superior.
2. RNF2 (Rendimiento): búsquedas < 2s en condiciones normales.
3. RNF3 (Seguridad): cifrado en tránsito (TLS) y cifrado en reposo.
4. RNF4 (Escalabilidad): soportar crecimiento de usuarios concurrentes.

5. RNF5 (Usabilidad): interfaz accesible y responsive.
6. RNF6 (Auditoría): log de operaciones críticas.
7. RNF7 (Backup/DR): respaldos diarios y pruebas de restauración.

15) Construir un diagrama E-R de un sistema de transporte urbano

Paso 1: Identificar entidades principales

- SERVICIO
- RUTA
- BUS
- CONDUCTOR
- PARADERO
- USUARIO

Paso 2: Definir atributos clave y de negocio

- SERVICIO: id_servicio (PK), tipo, horario, tarifa
- RUTA: id_ruta (PK), nombre, distancia
- BUS: id_bus (PK), placa, capacidad, id_ruta (FK)
- CONDUCTOR: id_conductor (PK), dni, nombre, licencia
- PARADERO: id_paradero (PK), nombre, ubicación
- USUARIO: id_usuario (PK), dni, nombre, tipo_usuario

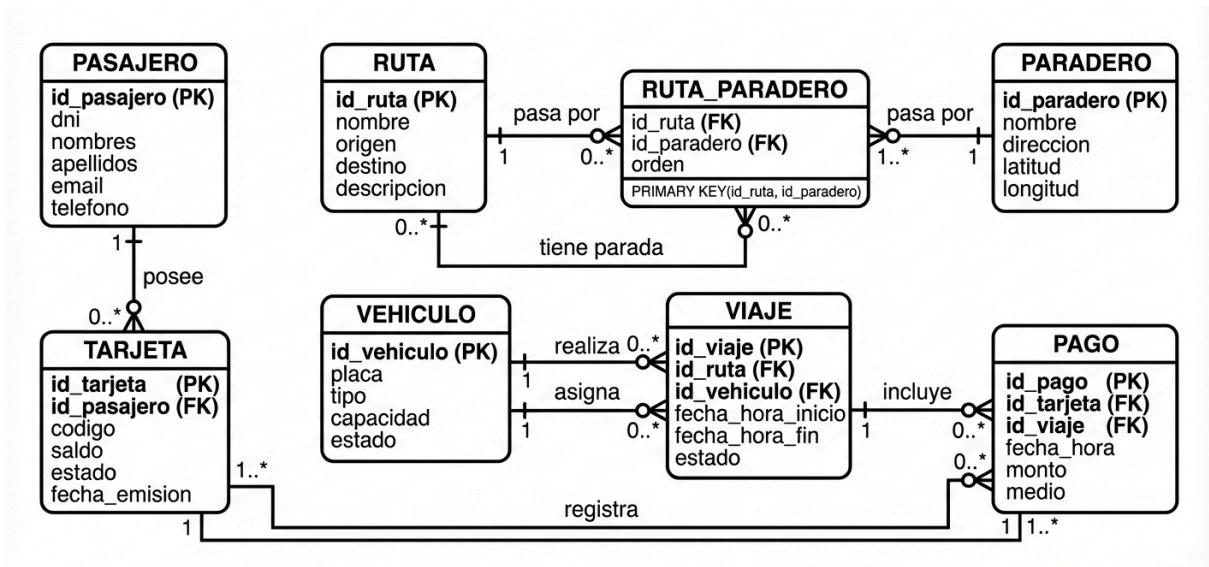
Paso 3: Definir claves primarias y foráneas (PK/FK)

- BUS.id_ruta **FK** hacia RUTA
- SERVICIO.id_bus **FK** hacia BUS
- SERVICIO.id_conductor **FK** hacia CONDUCTOR
- RUTA_PARADERO (tabla puente): id_ruta **FK**, id_paradero **FK**

Paso 4: Definir relaciones y cardinalidades

- RUTA 1..* BUS
- BUS 1..* SERVICIO
- CONDUCTOR 1..* SERVICIO
- RUTA .. PARADERO (muchos a muchos, vía tabla puente)

Paso 5: Resultado



16) Transformar un modelo lógico en tablas físicas con SQL en un DBMS

Paso 1: Analizar el modelo lógico del sistema de transporte urbano

Tablas principales: RUTA, BUS, CONDUCTOR, SERVICIO, PARADERO, USUARIO, RUTA_PARADERO (puente).

Paso 2: Definir restricciones y claves

- PK en cada tabla, FK donde corresponda
- UNIQUE en placa de BUS, dni de CONDUCTOR y USUARIO
- NOT NULL en campos clave
- CHECK en tipo_usuario y capacidad

Paso 3: Ejemplo de DDL SQL (dialecto genérico)

```

CREATE TABLE RUTA (
    id_ruta SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    distancia NUMERIC(5,2) NOT NULL
);
  
```

```

CREATE TABLE PARADERO (
    id_paradero SERIAL PRIMARY KEY,
  
```

```
    nombre VARCHAR(50) NOT NULL,  
    ubicacion VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE RUTA_PARADERO (  
    id_ruta INTEGER NOT NULL,  
    id_paradero INTEGER NOT NULL,  
    PRIMARY KEY(id_ruta, id_paradero),  
    FOREIGN KEY(id_ruta) REFERENCES RUTA(id_ruta),  
    FOREIGN KEY(id_paradero) REFERENCES PARADERO(id_paradero)  
);
```

```
CREATE TABLE BUS (  
    id_bus SERIAL PRIMARY KEY,  
    placa VARCHAR(10) UNIQUE NOT NULL,  
    capacidad INTEGER NOT NULL CHECK (capacidad > 0),  
    id_ruta INTEGER NOT NULL,  
    FOREIGN KEY(id_ruta) REFERENCES RUTA(id_ruta)  
);
```

```
CREATE TABLE CONDUCTOR (  
    id_conductor SERIAL PRIMARY KEY,  
    dni VARCHAR(12) UNIQUE NOT NULL,  
    nombre VARCHAR(60) NOT NULL,  
    licencia VARCHAR(20) NOT NULL  
);
```

```
CREATE TABLE USUARIO (  
    id_usuario SERIAL PRIMARY KEY,  
    dni VARCHAR(12) UNIQUE NOT NULL,  
    nombre VARCHAR(60) NOT NULL,  
    tipo_usuario VARCHAR(20) NOT NULL CHECK (tipo_usuario IN  
('adulto','escolar','senior'))  
);
```

```
CREATE TABLE SERVICIO (  
    id_servicio SERIAL PRIMARY KEY,  
    id_bus INTEGER NOT NULL,  
    id_conductor INTEGER NOT NULL,  
    horario VARCHAR(40) NOT NULL,  
    tarifa NUMERIC(5,2) NOT NULL CHECK (tarifa >= 0),  
    FOREIGN KEY(id_bus) REFERENCES BUS(id_bus),  
    FOREIGN KEY(id_conductor) REFERENCES CONDUCTOR(id_conductor)  
);
```

17) Proponer un plan de índices y claves para optimizar una base de datos de e-commerce

Paso 1: Identificar consultas típicas

- Por fecha (ventas del día, mes, etc.)
- Por cliente (historial, seguimiento)
- Por producto (stock, ventas por producto)
- Combinaciones (cliente+fecha, producto+fecha)

Paso 2: Proponer índices y claves

- Índices en fecha de pedidos (fecha_pedido)
- Índices en id_cliente y/o email
- Índices en id_producto
- Índice compuesto (id_cliente, fecha_pedido)
- UNIQUE en email del cliente

Paso 3: Consideraciones de costos y validación

- Evitar índices en campos con baja selectividad
- Balancear entre velocidad de consulta y costo de escritura
- Validar con EXPLAIN o EXPLAIN ANALYZE las consultas críticas

18) Comparar tres DBMS para una startup de software educativo

Paso 1: Candidatos

1. PostgreSQL
2. MySQL
3. MongoDB

Paso 2: Criterios de comparación

DBMS	Escalabilidad	Compatibilidad	Costo	Características técnicas	Tipo de datos
PostgreSQL	Alta (horizontal y vertical)	Muy compatible (SQL estándar)	Open source (sin costo de licencia)	Soporta transacciones, JSONB, extensiones	Relacional y semiestructurado
MySQL	Buena (especialmente vertical)	Muy compatible (SQL)	Open source (ediciones)	Transacciones, réplicas sencillas	Relacional

			Enterprise pagas)		
MongoDB	Alta (sharding nativo)	Menos compatible (NoSQL)	Open source, servicios gestionad os pagados	Flexible, escalable, sin joins complejos	Document os (JSON)

Paso 3: Recomendación

Para una startup educativa con requerimientos de integridad y consultas complejas, **PostgreSQL** suele ser la mejor opción; si la prioridad es velocidad y flexibilidad de datos, **MongoDB** es competitiva para casos no relacionales.

18) Justificar el uso de un DBMS relacional frente a uno NoSQL

Paso 1: Caso específico

Sistema académico universitario: matrículas, notas, historial de cursos.

Paso 2: Justificación paso a paso

- Modelo de datos estructurado, con relaciones y reglas de integridad
- Necesidad de transacciones (registro de matrícula, actualización de notas)
- Reportes complejos y consultas SQL
- Escenarios de auditoría y trazabilidad

Por lo anterior, se recomienda un **DBMS relacional** (PostgreSQL, MySQL).

¿Cuándo sería apropiado NoSQL?

- Si se requiere almacenar grandes volúmenes de datos semiestructurados (foros, logs, estadísticas de acceso)
 - Si la flexibilidad de esquema es prioritaria y no hay dependencias referenciales fuertes
-

19) Diseñar una tabla comparativa entre bases de datos locales y en la nube para un banco regional

Criterio	On-premise (local)	Cloud (gestionada)
Control	Total (hardware, datos)	Limitado (por el proveedor)
Costos	Inversión inicial alta, OPEX bajo	Pago por uso, OPEX variable
Disponibilidad	Depende de infraestructura propia	Alta (multi-zona, SLA)
Seguridad	Responsabilidad interna	Responsabilidad compartida
Cumplimiento	Fácil de personalizar	Debe validar compliance del proveedor
Escalabilidad	Limitada por hardware	Elástica, rápida
Latencia	Baja (usuarios locales)	Depende de red
Respaldo/DR	Manual, depende de TI	Automatizado, gestionado
Mantenimiento	Personal propio	Proveedor
Dependencia de proveedor	Baja	Alta

20) Documentar el proceso de instalación de un DBMS con capturas de pantalla

Paso a paso (ejemplo: PostgreSQL en Windows)

1. Descargar instalador oficial desde <https://www.postgresql.org/download/>.
2. **Captura 1:** Página de descarga seleccionada.

3. Ejecutar instalador. **Captura 2:** Inicio del instalador.
4. Seleccionar componentes (servidor, herramientas). **Captura 3:** Ventana de selección de componentes.
5. Elegir carpeta de instalación. **Captura 4:** Configuración de destino.
6. Configurar contraseña para el usuario "postgres". **Captura 5:** Pantalla de contraseña.
7. Elegir puerto (por defecto 5432). **Captura 6:** Selección de puerto.
8. Finalizar instalación. **Captura 7:** Pantalla de éxito.
9. Verificar servicio en ejecución (Panel de control o comando). **Captura 8:** Estado del servicio iniciado.

Checklist de evidencias

- Captura de descarga
- Captura de inicio de instalación
- Captura de selección de componentes
- Captura de contraseña
- Captura de finalización y verificación

21) Crear un script para configurar usuarios con roles específicos

Ejemplo en SQL (PostgreSQL)

-- Crear roles

```
CREATE ROLE admin LOGIN PASSWORD 'adminpass';
```

```
CREATE ROLE app_user LOGIN PASSWORD 'userpass';
```

```
CREATE ROLE auditor LOGIN PASSWORD 'auditpass';
```

-- Asignar privilegios mínimos

```
GRANT ALL PRIVILEGES ON DATABASE midb TO admin;
```

```
GRANT CONNECT ON DATABASE midb TO app_user;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO auditor;
```

-- Revocar privilegios (ejemplo)

REVOKE INSERT ON ALL TABLES IN SCHEMA public FROM auditor;

22) Simular una migración de un DBMS a otro (ejemplo: MySQL a PostgreSQL)

Paso a paso

1. Evaluar el esquema y las dependencias actuales en MySQL.
2. Exportar datos y esquema (mysqldump, CSV, etc.).
3. Transformar el esquema: ajustar tipos de datos y sintaxis para PostgreSQL.
4. Crear las tablas en PostgreSQL.
5. Cargar los datos en PostgreSQL (COPY, scripts adaptados).
6. Validar integridad y consistencia (consultas de conteo, constraints).
7. Realizar pruebas de aplicación y consultas.
8. Planificar el corte final (downtime mínimo).

23) Diseñar tablas para un sistema de reservas hoteleras

Entidades principales

- HABITACION (id_habitacion PK, numero, tipo, piso, estado)
- HUESPED (id_huesped PK, dni UNIQUE, nombre, email, telefono)
- RESERVA (id_reserva PK, id_huesped FK, id_habitacion FK, fecha_inicio, fecha_fin, estado)
- PAGO (id_pago PK, id_reserva FK, fecha_pago, monto, metodo)
- TARIFA (id_tarifa PK, id_habitacion FK, temporada, precio)

Regla para evitar solapamiento de reservas

Implementar una restricción lógica (triggers o validación en aplicación) para impedir que existan dos reservas activas para la misma habitación con rangos de fechas que se traslapan.

Estrategia: Al insertar o modificar una RESERVA, verificar que no exista otra con el mismo id_habitacion y fechas que se crucen.

24) Crear una vista para "habitaciones disponibles" en SQL

Ejemplo de VIEW

```
CREATE VIEW habitaciones_disponibles AS

SELECT h.id_habitacion, h.numero, h.tipo, h.piso

FROM habitacion h

WHERE h.estado = 'libre'

AND NOT EXISTS (

    SELECT 1 FROM reserva r

    WHERE r.id_habitacion = h.id_habitacion

    AND r.estado IN ('activa','confirmada')

    AND r.fecha_inicio <= CURRENT_DATE

    AND r.fecha_fin >= CURRENT_DATE

);
```

Consulta por rango de fechas

```
SELECT * FROM habitaciones_disponibles

WHERE id_habitacion NOT IN (

    SELECT id_habitacion FROM reserva

    WHERE estado IN ('activa','confirmada')

    AND fecha_inicio <= '2026-05-05'

    AND fecha_fin >= '2026-05-01'

);
```

25) Definir un índice en campo clave de búsqueda de clientes

Elección del campo

Por lo general, **dni** o **email** son los campos más usados para búsqueda de clientes.

Tipo de índice y ejemplo

-- Índice en dni

```
CREATE INDEX idx_cliente_dni ON cliente(dni);
```

-- O en email

```
CREATE INDEX idx_cliente_email ON cliente(email);
```

26) Establecer relaciones entre doctores y pacientes en un hospital

Tablas sugeridas

- MEDICO (id_medico PK, nombres, especialidad)
- PACIENTE (id_paciente PK, nombres, email, dni)
- CITA (id_cita PK, id_paciente FK, id_medico FK, fecha_hora, motivo)
- HISTORIA_CLINICA (id_historia PK, id_paciente FK UNIQUE, fecha_apertura)
- DIAGNOSTICO (id_diagnostico PK, id_historia FK, fecha, descripcion)

Cardinalidades

- Un médico puede tener muchas citas (1..*), cada cita corresponde a un paciente
- Un paciente puede tener muchas citas y una sola historia clínica
- Un diagnóstico pertenece a una historia clínica

27) Diseñar reglas de negocio mediante constraints en SQL

Ejemplos concretos

-- Estado de reserva solo puede ser 'activa', 'cancelada', 'finalizada'

```
ALTER TABLE reserva ADD CONSTRAINT chk_estado CHECK (estado IN ('activa','cancelada','finalizada'));
```

-- UNIQUE en email de huésped

```
ALTER TABLE huesped ADD CONSTRAINT uq_email UNIQUE (email);
```

-- FK con ON DELETE/UPDATE

```
ALTER TABLE reserva ADD CONSTRAINT fk_huesped FOREIGN KEY (id_huesped) REFERENCES huesped(id_huesped) ON DELETE CASCADE ON UPDATE CASCADE;
```

-- NOT NULL en campo obligatorio

```
ALTER TABLE habitacion ALTER COLUMN numero SET NOT NULL;
```

28) Poblar con datos ficticios un sistema de biblioteca universitaria

INSERTs de ejemplo

```
INSERT INTO usuario(id_usuario, nombre, email) VALUES (1, 'Ana Torres', 'ana@uni.edu');
```

```
INSERT INTO libro(id_libro, titulo, autor) VALUES (1, 'Base de Datos', 'Elena Ruiz');
```

```
INSERT INTO prestamo(id_prestamo, id_usuario, id_libro, fecha_prestamo, fecha_devolucion) VALUES (1, 1, 1, '2026-04-20', '2026-05-05');
```

La consistencia se asegura respetando las claves foráneas y restricciones de integridad.

29) Ejecutar consultas de prueba sobre disponibilidad de recursos

Ejemplos de consultas

-- Libros disponibles

```
SELECT * FROM libro WHERE id_libro NOT IN (SELECT id_libro FROM prestamo  
WHERE fecha_devolucion IS NULL);
```

-- Préstamos vigentes

```
SELECT * FROM prestamo WHERE fecha_devolucion IS NULL;
```

-- Top usuarios por número de préstamos

```
SELECT id_usuario, COUNT(*) AS total_prestamos FROM prestamo GROUP BY  
id_usuario ORDER BY total_prestamos DESC LIMIT 5;
```

30) Validar tiempos de respuesta de consultas en una base de datos grande

Metodología

- Generar datos sintéticos para tablas principales
- Crear índices en campos clave
- Ejecutar consultas con **EXPLAIN ANALYZE** para medir tiempos
- Registrar métricas (tiempo real, CPU, I/O)
- Definir umbrales aceptables (por ejemplo, <1s para búsquedas comunes)

31) Insertar registros inválidos en una base de datos académica y analizar el error

Objetivo

Comprobar el funcionamiento de restricciones de integridad insertando datos inválidos y documentando los errores generados por el DBMS.

Pasos

Identifica una tabla con restricciones (por ejemplo, `estudiante` con `UNIQUE(dni)` y `UNIQUE(email)`).

Ejecuta un `INSERT` que viole una restricción (por ejemplo, DNI repetido).

Registra el mensaje de error (captura o log).

Repite la prueba con otro tipo de restricción (email repetido, CHECK inválido, NOT NULL).

SQL

```
INSERT INTO estudiante (id_estudiante, dni, nombres, apellidos, email,
estado) VALUES

(10, '12345678', 'Juan', 'Pérez', 'juan@uni.edu', 'activo'),

(11, '12345678', 'Ana', 'Torres', 'ana@uni.edu', 'activo');
```

32) Probar restricciones de clave primaria y foránea en un sistema de matrículas

Objetivo

Validar que el DBMS impide registros huérfanos y duplicados en tablas relacionadas (por ejemplo `matricula`).

Pasos

Ubica las tablas `estudiante`, `seccion`, `matricula` y verifica sus PK y FK.

Intenta insertar una matrícula con un `id_estudiante` inexistente.

Intenta insertar una matrícula duplicada (si existe `UNIQUE(id_estudiante, id_seccion)`).

Registra los errores y valida que los datos consistentes siguen intactos.

SQL (ejemplo)

```
INSERT INTO matricula (id_matricula, id_estudiante, id_seccion,
fecha_matricula, estado)

VALUES (100, 999, 1, '2026-05-01', 'activa');
```

33) Diseñar un script que valide automáticamente integridad en una base de datos de ventas

Objetivo

Automatizar verificaciones de integridad referencial y consistencia lógica en un sistema de ventas.

Pasos

Define tablas clave: **cliente**, **venta**, **detalle_venta**, **producto**.

Crea consultas de validación (huérfanos, ventas sin detalle, montos inválidos).

Ejecuta el script y genera un reporte de resultados.

Programa su ejecución periódica (solo en entornos autorizados).

SQL

```
-- Ventas cuyo cliente no existe

SELECT v.id_venta

FROM venta v

LEFT JOIN cliente c ON v.id_cliente = c.id_cliente

WHERE c.id_cliente IS NULL;


-- Detalles cuyo producto no existe

SELECT d.id_detalle

FROM detalle_venta d

LEFT JOIN producto p ON d.id_producto = p.id_producto

WHERE p.id_producto IS NULL;


-- Ventas sin detalles

SELECT v.id_venta

FROM venta v
```

```
LEFT JOIN detalle_venta d ON v.id_venta = d.id_venta

GROUP BY v.id_venta

HAVING COUNT(d.id_detalle) = 0;
```

34) Medir el tiempo de respuesta de consultas simples y complejas en PostgreSQL

Objetivo

Medir y comparar el rendimiento de consultas simples vs complejas, y registrar tiempos.

Pasos

Prepara una base de datos de prueba con datos representativos.

Ejecuta una consulta simple y mide el tiempo.

Ejecuta una consulta compleja (joins y agregación) y mide el tiempo.

Usa **EXPLAIN ANALYZE** para obtener plan y tiempo total.

SQL (ejemplo)

```
SELECT * FROM cliente WHERE id_cliente = 1;

EXPLAIN ANALYZE

SELECT c.nombre, SUM(d.cantidad * d.precio_unitario) AS total

FROM cliente c

JOIN venta v ON c.id_cliente = v.id_cliente

JOIN detalle_venta d ON v.id_venta = d.id_venta

GROUP BY c.nombre

ORDER BY total DESC;
```

35) Simular 10 usuarios concurrentes en MySQL y medir la carga

Objetivo

Evaluar el comportamiento del DBMS bajo concurrencia moderada.

Pasos

Prepara una base de prueba con datos representativos.

Ejecuta una herramienta de carga (por ejemplo `mysqlslap` o `sysbench`).

Registra tiempo promedio y métricas (CPU, RAM, I/O, conexiones).

Comando de terminal (ejemplo)

```
mysqlslap --concurrency=10 --iterations=5 --query="SELECT * FROM producto  
WHERE precio > 1000;" --create-schema=midb --user=usuario --password=clave
```

36) Comparar el rendimiento de una consulta antes y después de la creación de un índice

Objetivo

Demostrar el impacto de un índice sobre una consulta típica.

Pasos

Ejecuta la consulta sin índice y registra el plan/tiempo.

Crea el índice.

Ejecuta la misma consulta y compara plan/tiempo.

SQL (ejemplo)

```
EXPLAIN ANALYZE
```

```
SELECT * FROM venta WHERE fecha_venta = '2026-04-28';
```

```
CREATE INDEX idx_fecha_venta ON venta(fecha_venta);
```

```
EXPLAIN ANALYZE
```

```
SELECT * FROM venta WHERE fecha_venta = '2026-04-28';
```

Evidencias

Capturas de ambos planes.

Resultados esperados

Menor tiempo y cambio del plan (por ejemplo, index scan).

Advertencia: índices extra impactan inserciones/actualizaciones, evalúa costo-beneficio.

37) Detectar consultas lentas en un sistema académico y reescribirlas

Objetivo

Identificar una consulta lenta y mejorarla reescribiéndola o agregando índices.

Pasos

Activa log de consultas lentas o identifica consultas que tardan más.

Revisa el plan con **EXPLAIN**.

Reescribe la consulta o agrega un índice compuesto.

Repite la medición.

SQL (ejemplo)

```
EXPLAIN ANALYZE

SELECT *

FROM matricula

WHERE estado = 'activa'

AND fecha_matricula < '2026-04-01';
```

```
CREATE INDEX idx_matricula_estado_fecha ON matricula(estado,
fecha_matricula);
```

38) Crear un plan de optimización para un sistema de biblioteca digital

Objetivo

Definir un plan de mejoras para rendimiento y mantenimiento del sistema.

Pasos

Lista consultas frecuentes (búsqueda, préstamos vigentes, historial).

Propón índices (por ejemplo en `titulo`, `autor`, `id_usuario`).

Considera archivar históricos y/o particionar tablas muy grandes.

Define monitoreo (slow queries, alertas) y estrategia de mantenimiento.

Resultados esperados

Reducción de tiempos en consultas frecuentes y mejor estabilidad.

39) Simular un aumento de carga en SQL Server y proponer ajustes en índices y servidor

Objetivo

Evaluar el comportamiento bajo carga alta y proponer ajustes de performance.

Pasos

Genera carga con una herramienta (por ejemplo SQLQueryStress o HammerDB).

Mide tiempos y recursos (CPU, memoria, I/O).

Identifica cuellos: consultas, bloqueos, falta de índices.

Propón cambios: índices, configuración, memoria, paralelismo.

Evidencias

Reporte de métricas y recomendaciones.

Resultados esperados

Acciones concretas para bajar latencia y reducir bloqueos.

40) Configurar un entorno de pruebas y registrar el tiempo de ejecución de 50 consultas complejas

Objetivo

Establecer una línea base de rendimiento con un conjunto de consultas exigentes.

Pasos

Crea un entorno de prueba con datos representativos.

Define 50 consultas complejas.

Ejecuta y registra tiempos por consulta.

Identifica las consultas más costosas.

Evidencias

Tabla de benchmarking completada.

Resultados esperados

Ranking de consultas por tiempo y posibles optimizaciones.

41) Comparar el throughput de MySQL vs PostgreSQL usando los mismos datos

Objetivo

Comparar consultas por segundo y latencia promedio bajo condiciones equivalentes.

Pasos

Monta ambos DBMS con mismo esquema y volumen de datos.

Usa herramientas equivalentes (por ejemplo sysbench y pgbench).

Ejecuta pruebas con mismo número de hilos, duración y tipo de consulta.

Registra throughput y latencias.

Evidencias

Resultados de herramientas y tabla comparativa.

Resultados esperados

Comparación objetiva y explicación de diferencias.

42) Analizar uso de CPU y memoria en un DBMS durante 1 hora de operación continua

Objetivo

Observar consumo de recursos en operación sostenida y detectar saturación.

Pasos

Configura monitoreo (top/htop/perfmon, métricas del DBMS).

Ejecuta carga estable por 1 hora.

Registra cada 5 o 10 minutos: CPU, RAM, I/O, conexiones.

Identifica picos y causas (consultas, bloqueos, falta de índices).

Evidencias

Tabla de monitoreo completada y gráficas.

Resultados esperados

Hallazgos y recomendaciones (más recursos, tuning, índices).

43) Redactar un reporte técnico con métricas de rendimiento obtenidas en una base de datos de hospital

Objetivo

Elaborar un reporte técnico que documente las principales métricas de rendimiento (tiempos de consulta, uso de CPU, I/O, bloqueos, etc.) obtenidas en la base de datos de un hospital, siguiendo una estructura profesional.

Pasos

Identifica las consultas y procesos críticos del sistema hospitalario (admisión, consulta, alta, facturación, etc.).

Recolecta métricas de rendimiento usando herramientas como **EXPLAIN ANALYZE**, monitoreo del servidor o logs.

Registra los principales indicadores: tiempo de respuesta, consumo de CPU/RAM, número de conexiones, bloqueos detectados.

Organiza la información en el reporte con secciones claras.

Plantilla de reporte técnico

Sección	Contenido esperado
Resumen ejecutivo	Descripción breve de objetivos y hallazgos principales

Metodología	Herramientas y procedimientos usados para obtener métricas
Métricas recolectadas	Tabla y análisis de tiempos, CPU, bloqueos, etc.
Análisis	Interpretación de los resultados, identificación de problemas
Conclusiones	Resumen de hallazgos clave

Evidencias

Capturas de pantallas de métricas y logs.

Tabla consolidada de resultados.

Resultados esperados

Reporte técnico completo y estructurado, listo para ser presentado a la gerencia o área técnica.

Advertencia: protege datos sensibles y personales en los reportes.

44) Elaborar gráficos comparativos sobre el tiempo de ejecución de consultas antes y después de aplicar optimizaciones

Objetivo

Visualizar el impacto de las optimizaciones realizadas en consultas SQL, comparando los tiempos de ejecución antes y después mediante gráficos.

Pasos

Selecciona una o varias consultas representativas del sistema.

Ejecuta cada consulta antes de optimizar y registra los tiempos (usando **EXPLAIN ANALYZE** u otra herramienta).

Aplica la optimización (creación de índices, reescritura de la consulta, etc.).

Vuelve a ejecutar las consultas y registra los nuevos tiempos.

Completa una tabla comparativa con ambos resultados.

Usa los datos para elaborar un gráfico de barras (puede ser dibujado a mano o en hoja de cálculo).

Ejemplo de tabla para registrar tiempos

Consulta	Tiempo antes (ms)	Tiempo después (ms)	Notas
Consulta A			
Consulta B			

Para graficar: toma los datos de la tabla y representa cada consulta con dos barras (antes y después), mostrando visualmente la mejora.

Evidencias

Tabla de tiempos antes y después.

Gráficos comparativos (foto, captura o exportado).

Resultados esperados

Reducción significativa de tiempos visibles en el gráfico.

Advertencia: asegúrate de que las mediciones se hagan en condiciones equivalentes.

45) Diseñar un reporte con conclusiones y recomendaciones para mejorar el rendimiento en un sistema de ventas

Objetivo

Elaborar un reporte profesional que presente conclusiones y recomendaciones concretas para optimizar el rendimiento de una base de datos de ventas.

Pasos

Analiza los principales problemas detectados (consultas lentas, bloqueos, saturación de recursos).

Redacta conclusiones basadas en la evidencia recopilada.

Propón recomendaciones prácticas y priorizadas (índices, normalización, particionamiento, hardware, etc.).

Organiza el reporte siguiendo una estructura clara.

Plantilla de reporte de recomendaciones

Sección	Contenido esperado
Introducción	Contexto del sistema de ventas
Conclusiones	Resumen de hallazgos principales
Recomendaciones	Lista detallada y justificada de acciones sugeridas
Prioridad	Orden de implementación sugerido
Anexos	Datos de soporte, capturas, planes de consulta

Evidencias

Documento con las conclusiones y recomendaciones.

Resultados esperados

Reporte claro, accionable y orientado a la mejora continua.

Advertencia: toda recomendación debe ser validada antes de aplicarse en producción.

46) Detectar un cuello de botella en una base de datos de transporte y proponer una mejora

Objetivo

Identificar una consulta lenta en el modelo de Transporte Urbano (tablas RUTA, VIAJE, PAGO, TARJETA, etc.), analizar su causa y proponer una optimización efectiva.

Pasos

Selecciona una consulta realista y costosa, por ejemplo:

```
SELECT r.nombre_ruta, COUNT(p.id_pago)
FROM ruta r
JOIN viaje v ON r.id_ruta = v.id_ruta
JOIN pago p ON v.id_viaje = p.id_viaje
WHERE p.fecha_pago BETWEEN '2026-04-01' AND '2026-04-30'
      AND p.monto > 5
GROUP BY r.nombre_ruta
      ORDER BY COUNT(p.id_pago) DESC;
```

Ejecuta la consulta y revisa el plan (**EXPLAIN ANALYZE**), observando si hay *full table scan* o joins lentos.

Identifica la causa: falta de índices compuestos, joins no optimizados, filtros sobre campos sin índice.

Propón una mejora, por ejemplo crear un índice compuesto:

```
CREATE INDEX idx_pago_fecha_monto ON pago(fecha_pago, monto);
```

Vuelve a ejecutar la consulta y compara el plan/tiempo.

Documenta el análisis antes/después y justifica la mejora.

Evidencias

Planes de ejecución antes y después.

Captura del tiempo de respuesta.

Resultados esperados

Consulta optimizada con menor tiempo y uso de índices compuestos.

Advertencia: toda optimización debe ser probada primero en entornos de desarrollo.

47) Implementar un ciclo PDCA en un proyecto de base de datos académica

Objetivo

Aplicar el ciclo de mejora continua PDCA (Plan-Do-Check-Act) para optimizar el rendimiento de una base de datos académica, definiendo actividades y entregables por fase.

Pasos

Plan: Diagnóstico inicial, definición de objetivos y selección de métricas a mejorar (ejemplo: reducir tiempos de consulta en matrícula).

Do: Implementación de acciones (creación de índices, refactorización de consultas, ajustes de configuración).

Check: Medición de resultados (comparación de métricas antes/después, análisis de logs).

Act: Estandarización de mejoras efectivas y planificación de nuevas acciones si es necesario.

Entregables por fase

Plan: Documento de diagnóstico y objetivos.

Do: Registro de tareas ejecutadas (scripts, cambios, responsables).

Check: Informe de comparación de indicadores.

Act: Procedimientos actualizados y plan de seguimiento.

Cronograma sugerido (2 semanas)

Semana	Fase	Entregable
1	Plan/Do	Diagnóstico, plan de acción, ejecución de mejoras iniciales
2	Check/Act	Medición, informe de resultados, actualización de procedimientos

Evidencias

Documentos de cada fase y cronograma cumplido.

Resultados esperados

Mejora continua y documentada en el rendimiento de la base de datos.

Advertencia: documenta todos los cambios y respalda la información antes de modificarlos.

48) Rediseñar una consulta SQL ineficiente y demostrar la mejora obtenida tras la optimización

Objetivo

Transformar una consulta SQL con anti-patrones (por ejemplo, `SELECT *` y subconsultas no indexadas) en una versión optimizada, demostrando la mejora con `EXPLAIN ANALYZE`.

Pasos

Identifica una consulta ineficiente, por ejemplo:

```
SELECT *
FROM estudiante
WHERE id_estudiante IN (
    SELECT id_estudiante FROM matricula WHERE estado = 'activa'
);
```

Ejecuta la consulta y registra el plan/tiempo con `EXPLAIN ANALYZE`.

Rediseña la consulta:

```
SELECT e.id_estudiante, e.nombres, e.apellidos
FROM estudiante e
JOIN matricula m ON e.id_estudiante = m.id_estudiante
WHERE m.estado = 'activa';
```

Crea los índices necesarios (por ejemplo, `CREATE INDEX idx_matricula_estado ON matricula(estado);`).

Vuelve a medir con `EXPLAIN ANALYZE` y compara resultados.

Evidencias

Planes de ejecución y tiempos antes/después.

Capturas de resultados y cambios aplicados.

Resultados esperados

Consulta optimizada y justificación técnica de la mejora obtenida.